

Ontology Drift, Schema Evolution, and Reality Drift

Why a Knowledge System Can Remain Consistent While Its World Changes

Knowledge Systems and Reality Drift - Note #1 v1

A. Jacobs - Semantic Fidelity Project

The Basic Pattern

Knowledge systems depend on stable categories. Databases, knowledge graphs, medical records, regulatory systems, and artificial intelligence all need defined entities, properties, and relationships before information can be stored or reasoned over. An ontology supplies this structure. It specifies what kinds of things exist in a domain and how those things may relate.

That structure is useful because reality is too variable to manage case by case. But the domain does not stop changing after its ontology is built. New practices emerge, old distinctions weaken, entities acquire new roles, and previously peripheral cases become important. The formal model may remain logically consistent even as its fit with the world deteriorates.

Ontology drift begins in this gap. The system continues accepting data, validating records, and returning answers. Nothing has to crash. The ontology can work exactly as designed while progressively misdescribing the reality it organizes.

When the Schema Outlives the Domain

Ontology evolution is the deliberate modification of concepts, relations, and constraints as a domain changes. Ontology drift is the condition that makes such modification necessary: the conceptual structure and the represented domain no longer change together.

At first, the mismatch is usually small. New cases can be handled through exceptions, free-text notes, improvised mappings, or broad residual categories. These repairs preserve operation without revising the underlying model. Their success therefore hides the accumulating misfit.

Once institutional processes, software integrations, reporting rules, and trained models depend on the inherited schema, revision becomes costly. The representation gains infrastructural weight. Reality is increasingly translated to fit the ontology because changing the ontology would disturb everything built above it.

How Ontology Drift Emerges

Stage 1 - Formalization

A domain is translated into classes, properties, relations, and constraints. The ontology captures enough of the current environment to support coordination.

Stage 2 - Stabilization

Applications, records, workflows, and decisions become dependent on the schema. Its categories acquire operational authority.

Stage 3 - Domain Change

Practices, entities, meanings, and relationships change. New cases no longer fit the original conceptualization cleanly.

Stage 4 - Accommodation

Users preserve continuity through exceptions, miscellaneous categories, mappings, and local workarounds. These absorb evidence that the ontology needs revision.

Stage 5 - Drift

The knowledge system remains valid according to its own rules while becoming less answerable to the domain those rules were meant to represent.

Reality Drift and Representational Lag

Reality Drift occurs when a system remains operational while losing alignment with the conditions it was built to reflect or improve. Ontology drift is a formal instance of the same pattern. Internal consistency measures whether statements conform to the ontology. It does not establish whether the ontology still preserves the important structure of the world.

The crucial variable is representational lag: the distance between change in the domain and change in the model. Versioning can record revisions, but it does not guarantee that revision occurs soon enough, reaches dependent systems, or restores distinctions lost during the period of mismatch.

A knowledge system remains grounded only when omitted or anomalous cases can return as pressure on the ontology itself. If every anomaly is forced into an inherited category, the system learns that its schema is complete precisely because contrary evidence has nowhere to appear.

The Representation Stack

Ontology drift compounds as the schema moves upward through a representation stack. A changing condition becomes a record, the record is classified by an inherited schema, the classified data becomes a metric, and the metric becomes evidence for policy or automated action.

Each layer may be technically correct relative to the layer below it. The failure lies in the original translation between the domain and the ontology. Once that mismatch has propagated, downstream agreement can make the representation appear increasingly authoritative while contact with the changing world weakens.

Examples Across Systems

Healthcare: Diagnostic and billing ontologies may preserve categories that no longer match clinical practice, forcing new conditions and ambiguous cases into administratively stable classifications.

Organizations: A customer, product, or risk taxonomy may reflect an earlier business model while dashboards and decisions continue treating those categories as current.

Public Administration: Eligibility systems may rely on household, employment, or disability categories that fail to capture newer forms of work and dependency.

Artificial Intelligence: A knowledge graph can preserve obsolete entity types and relations while retrieval and reasoning remain internally coherent.

Conclusion

Ontology drift is not simply outdated documentation. It is the persistence of a conceptual structure after its original fit with the world has weakened. Because the system remains consistent, searchable, and operational, the mismatch can accumulate without appearing as failure.

The corrective task is not constant redesign. It is maintaining a live path through which new cases, exceptions, and changed relationships can revise the conceptual model. The absence of contradiction is not proof of ontological alignment.

Related Phrases and Concepts

- ontology drift
- ontology evolution
- schema evolution
- concept drift
- semantic drift
- ontology versioning
- dynamic knowledge graphs
- representational lag

Semantic Fidelity Project Resources

- [Research Library \(GitHub\)](#)
- [Articles & Essays \(Substack\)](#)
- [Semantic Fidelity Glossary](#)